ChatGPT

# BranchWrite as a Test-Time-Compute-Optimal Runtime

The central conclusion from the research is that **the durable typed call-frame design is still one abstraction too low and one abstraction too sequential**.

It solves an important systems problem—persistent calls, suspended parents, typed returns, crash recovery—but it quietly assumes the shape of computation is fundamentally:

```
parent
  calls child
    child returns
  parent resumes
```

That is not the shape we want to optimize.

For BranchWrite, the more powerful formulation is:

> **A run is an anytime, budgeted search over persistent candidate futures.**

A call stack is one possible projection of that search. It is not the state model.

That change matters because research on test-time scaling increasingly points in the same direction: the best use of additional inference is **not uniform extra thinking**. It depends on task difficulty, remaining budget, uncertainty, candidate quality, verifier quality, and what has already been learned. Snell et al. found adaptive compute allocation substantially more efficient than naïve best-of-N; newer budget-aware tree search explicitly changes strategy as the remaining token budget shrinks; long-horizon coding research argues that scaling becomes a problem of representing, selecting, and reusing previous trajectories rather than merely generating more of them. [1]

The BW v0.4 architecture already has several unusually important pieces pointing in the right direction: persistent COW model state, explicit physical-world state, causal history distinct from UI projection, independent planning branches, isolated execution branches, evidence-bound acceptance, and uncertainty-preserving recovery. filecite turn0file0 The native COW interface is especially valuable because a fork really is a persistent model-context operation; committed partial generations and active turns are distinct; `continue` resumes an existing assistant turn; and an uncertain request can be recovered from its original operation identity rather than reconstructed as a new chat request. filecite turn0file2

The design change I would make is therefore not to bolt a smarter scheduler onto the previous call-frame design. It is to **redefine the substrate so that search, reuse, verification, suspension, and recovery all fall out of one persistent computation model.**

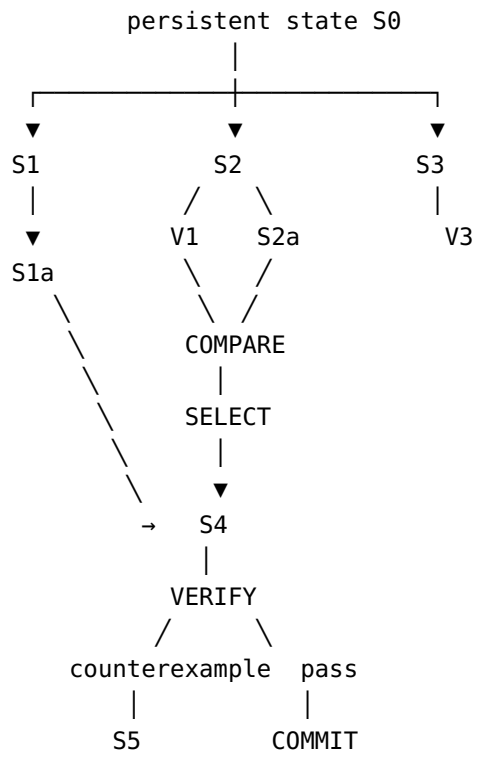
Download the full implementation handoff

## The architectural correction

The previous thinking was:

```
typed invocation
    owns frame
    calls child
    suspends
    child returns typed value
    result delivered to parent slot
    child pops
    parent resumes
```

The new thinking should be:



Here:

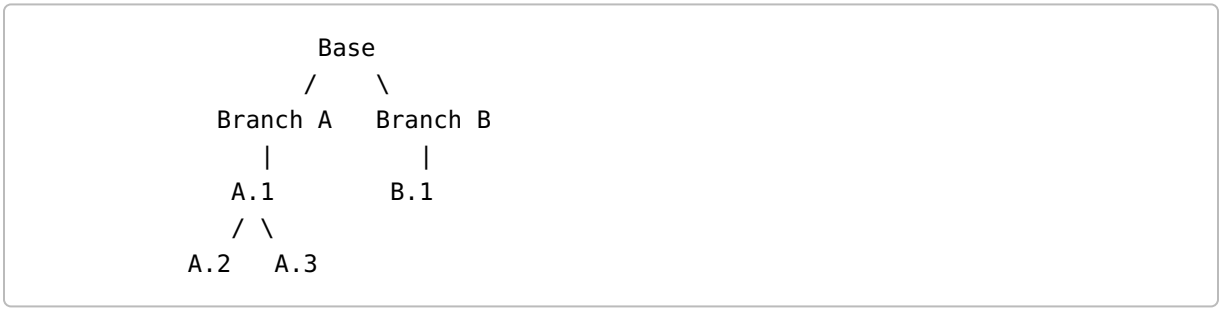
- `S*` are durable **state capsules**.
- Edges are typed computations.
- `V*` are evidence or derived values.
- The active work is a **frontier**, not a stack pointer.
- A state can have many descendants.
- Results can have several consumers.
- A verifier can attack an implementation rather than merely "return" to it.
- Selection can retire dozens of branches without deleting them.
- A synthesis can consume multiple branches without pretending their model KV state was merged.
- A crash does not require reconstruction of an ephemeral stack.

This is consistent with the broader search literature. Language Agent Tree Search demonstrated the value of combining reasoning, acting, planning, and environmental feedback inside tree search, while Graph of Thoughts showed why useful information flow eventually exceeds a strict single-parent thought tree: comparison, aggregation, and distillation naturally form graph dependencies. Dynamic population methods likewise use selection and resampling rather than uniformly extending every branch. [2]

## The tree remains fundamental—but it is not the entire graph

There are really two structures hiding inside "the tree."

**The first is a true COW worldline tree.**

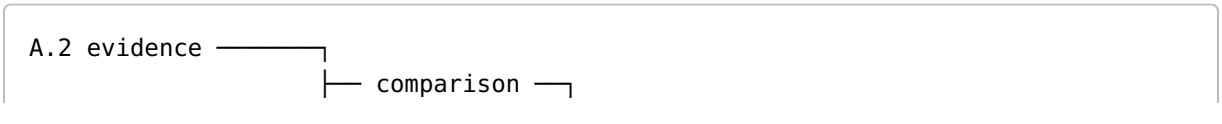



This captures inheritance:

- model-context ancestry;
- filesystem/world ancestry;
- immutable shared prefixes;
- isolation of subsequent mutation.

Each child has one actual COW parent for a particular state dimension.

**The second is a knowledge/evidence graph.**



```
B.1 evidence ──────┘                │
                                    ├── selection
A.3 counterexample ─────────────────┘
```

Knowledge can have multiple parents.

Trying to force this second structure into a tree creates artificial copying, hidden state propagation, or fake "returns." Conversely, turning the underlying COW lineage itself into an arbitrary graph obscures what was genuinely inherited from what.

So I would make this a hard BW distinction:

> **Physical and cognitive inheritance is tree-shaped where COW requires a parent.**
> **Knowledge composition is DAG/hypergraph-shaped.**

That is more precise than saying "BW has a tree."

### The runtime state

At the highest level:

```
Run = (G, F, I, B, P)
```

where:

```
G = immutable typed causal hypergraph
F = currently eligible search frontier
I = current incumbent candidate set
B = remaining multidimensional budget
P = policy for deciding where the next compute goes
```

This is the abstraction I would expect to survive even if every primitive underneath BW were radically better 100 years from now.

Model architecture can change.

Execution substrate can change.

Filesystem can disappear.

GPU KV caches can become something else.

But the problem remains:

> Given what I currently know, what candidate futures still exist, what evidence differentiates them, and what finite computation should I perform next to maximize the quality of my final decision?

That is the stable abstraction.

## The tree should allocate compute, not merely organize it

The strongest research result for BW is not simply “tree search works.” It is that **the shape of the search should depend on what is happening during the run**.

Snell et al. found that the best test-time strategy varies by problem difficulty: easier problems can favor sequential refinement while more difficult problems can benefit more from parallel exploration or search, and adaptive allocation achieved over fourfold efficiency relative to their best-of-N baseline in studied settings. [3]

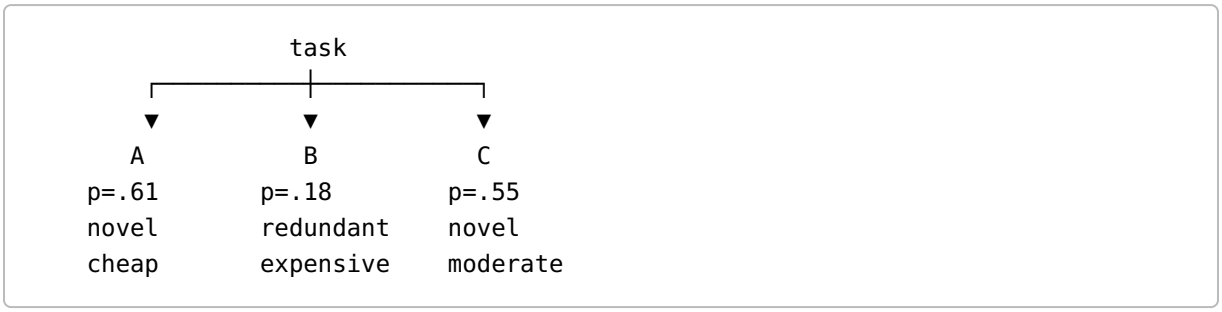
Budget-Guided MCTS goes further: under a fixed token budget, it deliberately explores more broadly when budget is plentiful, then reduces late branching and shifts toward refinement/completion as the budget runs down. [4] HyPER similarly frames test-time inference as an online **expand–reduce** control problem over a population of hypothesis paths rather than a fixed-depth tree. [5]

For long-running agents, blindly multiplying trajectories is especially weak. The 2026 agentic-coding scaling work argues that prior rollout experience has to be represented compactly enough to select and reuse; its parallel method recursively compares rollout summaries, while its sequential method distills previous experience into future attempts. [6]

That suggests a very different BW scheduler.

### Branches do not deserve equal compute

Suppose we have:




The scheduler should not say:

> “Each branch gets another Builder turn.”

It should ask:

> “What computation has the greatest expected marginal effect on the final decision?”

Candidate computations could include:

```
extend A
challenge A
fork A at its uncertain architectural choice
abandon B
compare A against C
run a cheap test on C
generate another fundamentally different approach
distill what A and C agree on
investigate one unresolved repository fact
```

The allocation target is therefore approximately **expected value of computation**.

Conceptually:

$$\text{priority}(c) = E[\Delta U_{\text{final}} \mid c] + E[\text{information gain} \mid c] - \lambda\, E[\text{resource cost} \mid c] - \text{risk}(c) - \text{redundancy}(c) + \text{locality b}$$

It need not literally use that formula in v1. The important architectural property is that **the policy interface has enough information to eventually approximate it**.

Classical metareasoning treats computation itself as a decision: spend another computation only when its expected information or decision value warrants its cost. Search theory also distinguishes *simple regret*—the quality of the final action—from cumulative reward collected while exploring; that distinction maps unusually well to software work, because a beautiful losing candidate gives us value only through what it teaches the final selection. [7]

### BW should be anytime

There should always be an incumbent:

```
Best current candidate:
    candidate-17

Known to pass:
    typecheck
    unit tests
    parser regression tests

Not yet established:
    concurrency regression
    full integration suite
```

```
Known weakness:
    verifier-9 found a timeout edge case,
    candidate-17b contains proposed repair

Expected best next computation:
    execute candidate-17b against targeted timeout test
```

That means killing BW at any time still yields:

1. the best candidate currently justified;
2. its exact evidence;
3. what remains unknown;
4. which extra computation BW would have purchased next and why.

That is much stronger than “there is currently a parent call waiting on child 42.”

### Branch factor should be endogenous

BW should expand harder when uncertainty is high.

It should branch less when alternatives collapse onto equivalent states.

It should abandon branches whose marginal expected value falls below the current price of compute.

It should allocate verifier compute when **selection uncertainty**, rather than generator uncertainty, becomes dominant.

Recent agentic test-time-scaling work supports this kind of nonuniform allocation: confidence-aware WebAgent scaling uses uncertainty signals to selectively spend extra compute rather than uniformly multiplying samples, and Conformal Thinking treats stopping and abandonment themselves as budget/risk decisions. [8]

So something like:

```
high uncertainty + weak incumbent
    -> EXPAND

promising branch + localized defect
    -> REFINE

several plausible candidates
    -> COMPARE

apparently-good incumbent + weak verification
    -> CHALLENGE
```

```
strong incumbent + low marginal search value
    -> VERIFY deeply

acceptance proven + further EVC below threshold
    -> COMMIT
```

Those are **policies over the same substrate**, not separate agent architectures.

### COW topology itself should influence the scheduler

This is where BW can exploit your tree more deeply than ordinary agent systems.

Two candidate generations with 2,000 output tokens are not necessarily the same cost.

Consider:

```
                COW snapshot X
               /      |       \
             A        B        C
```

If X's shared prefix/KV state is resident, expanding A, B, and C together may be drastically cheaper than independently rebuilding that context.

SGLang's RadixAttention demonstrates the importance of reusing shared KV prefixes across structured language-model programs. [9] Your existing COW semantics make this even more directly expressible: branches fork from a saved context, and the native interface distinguishes real branch/snapshot/revision identity rather than merely sending a reconstructed transcript. filecite turn0file2

Therefore the scheduler should know:

```
semantic value of computation
+
actual marginal inference cost
+
prefix residency / reuse
+
checkpoint migration cost
+
memory pressure / eviction likelihood
```

Not merely:

```
number_of_tokens
```

That produces a very interesting optimization:

> **Search topology and execution topology become jointly optimizable without becoming semantically conflated.**

A warm sibling must not outrank an enormously more valuable branch merely because it is cheaper. But between roughly equivalent next moves, BW should exploit the warm COW family.

Eventually the runtime could support:

```
FORK 16 FROM snapshot-X
GENERATE alternatives AS one prefix-sharing expansion family
```

while still recording sixteen independent descendant contexts.

## Typed calls become graph computations

The type system is still essential. What changes is where it lives.

Rather than:

```
Frame<Builder>
  local variables
  return slot
  child frame
```

I would define:

```
Computation<I, O>
```

where an immutable computation consumes typed inputs and establishes typed outputs.

For example:

```
Expand<State, CandidateSet>
Observe<WorldState, FileObservation>
Act<PatchIntent, ExecutionObservation>
Verify<Candidate, VerificationEvidence>
Challenge<Candidate, CounterexampleEvidence>
```

```
Compare<CandidateSet, Comparison>
Select<CandidateSet, SelectionDecision>
Distill<BranchSet, BranchDigest>
Commit<Candidate, PublicationReceipt>
```

Typed effects and handlers are a good theoretical foundation here because they cleanly separate *the request for an effect* from *the handler that decides how that effect is realized*, while retaining resumable control semantics. Work on algebraic effects explicitly connects this model with delimited continuations, backtracking, cooperative concurrency, and other nontrivial control structures. [10]

### The most important simplification: remove mutable return delivery

This is where I would deliberately **discard part of the previous v0.5 design**.

Do not represent this:

```
child finished
    ↓
deliver result to parent's slot
    ↓
mark slot delivered
    ↓
pop child
    ↓
resume parent
```

Represent this:

```
C_child
    └── produces Result<R>

C_next
    └── requires Result<R> from C_child
```

When `C_child` has a durably committed result, the dependency of `C_next` is satisfied.

**Readiness is derived.**

There is no separate “delivery” mutation.

There is no global `last_result`.

There is no delivery bit.

There is nothing to pop.

There is no question about whether a child result had been copied into a parent frame before the crash.

This is the biggest architectural improvement I see from re-deriving the design.

Structured concurrency remains useful as an execution discipline—scoped children, explicit joins, cancellation when a join condition is reached—and contemporary `StructuredTaskScope` designs make different join policies explicit. [11] But **structured concurrency should govern live resources; it should not define BW's durable semantic history**.

A sequential function-like flow simply compiles to:

```
S0
 |
 C1
 | result A
 ▼
C2 requires A
 |
 result B
 ▼
C3 requires A + B
```

A parallel flow becomes:

```
            S0
          /   |   \
        C1   C2   C3
         |    |    |
        R1   R2   R3
          \   |   /
            JOIN
        requires R1,R2,R3
```

“First successful candidate wins” is another dependency/join policy.

“Wait for any three of ten.”

“Wait until confidence > threshold.”

“Wait until a counterexample exists.”

“Wait until all mandatory complementary branches complete.”

They are all predicates over durable graph facts.

### What persists when a branch “unwinds”

Nothing moves.

Nothing is copied into a second history tree.

Nothing disappears.

Suppose:

```
task
└── candidate-A
    └── tests
        └── targeted-regression
```

After `targeted-regression` completes, the *active frontier* might become:

```
candidate-A / interpret-regression-result
```

while history still contains:

```
candidate-A
└── tests
    └── targeted-regression
        ├── plan
        ├── authorization
        ├── process execution
        ├── stdout artifact
        ├── stderr artifact
        ├── result
        └── provenance
```

So “pop” is purely a UI metaphor for:

> this node is no longer on the live frontier.

The COW chat system already exhibits an analogous persistence principle: a rewind moves the selected branch tip but saved history remains recoverable, and rewinding conversation state explicitly does not undo external tool effects. 𝌆filecite𝌆turn0file1𝌆

BW should generalize that principle to its whole computation universe.

### A result can have several consumers

This becomes extremely important for test-time scaling.

One failed execution result might feed:

```
failure
├── Builder refinement A
├── Builder refinement B
├── root-cause-analysis branch
├── test-generator branch
├── summary generator
└── candidate comparator
```

A call-stack architecture treats multiple consumers as awkward fan-out.

A persistent graph considers it normal.

### Multi-parent synthesis is explicit

Suppose A discovered a robust cache invalidation strategy and B found a race condition A missed.

We should not say:

> “Merge A's model context into B.”

There generally is no principled operation corresponding to that.

Instead:

```
BranchDigest(A) ──────┐
A artifacts ──────────┤
                      ├── SYNTHESIZE
BranchDigest(B) ──────┤      │
B counterexample ─────┘      ▼
                        Context C
```

`Context C` is a **new cognitive lineage** seeded from selected artifacts.

That preserves honesty about what COW actually does.

Graph-of-Thought-style composition supports the conceptual point that useful reasoning dependencies can be multi-parent even when underlying generation remains sequential within each context. [12]

## Headlines, summaries, reasoning, and tools become typed side products

This is where your earlier question about “all the Codex-like stuff” gets a cleaner answer.

Those features should not merely “stack onto the call tree.”

They should attach to the **semantic graph according to what they actually are**.

### Headlines

A headline is:

```
Annotation<Headline> {
    text
    source_state
    generator
    template_version
    creation_operation
}
```

Graphically:

```
               state-14
               /     \
              /       \
      continue work   GENERATE_HEADLINE
                           |
                     headline-14
```

The headline computation does not contaminate the coding context unless another explicit transition chooses to inject it there.

A late headline generated for state 14 does not become the title of state 22 by accident.

A user-edited title gets separate provenance and an explicit precedence rule.

### Status

There are two different things:

```
CanonicalStatus
```

and

```
StatusNarration
```

Canonical status is computed mechanically:

```
frontier branches: 6
live computations: 3
current incumbent: candidate-21
mandatory checks satisfied: 5 / 7
unresolved effects: 0
remaining model budget: ...
```

No model generation is required.

A language model may derive:

> “Two implementation families remain; the stronger candidate is waiting on integration verification.”

That is useful UX, but it remains an annotation.

**A prose status message never becomes workflow truth.**

### Summaries become search infrastructure

Summaries are more important than headlines because they can improve test-time compute allocation.

The agentic-coding scaling work specifically reports that compact structured summaries of rollout trajectories can preserve hypotheses, progress, and failure modes sufficiently to support tournament selection and reuse of previous attempts. [6]

BW should take that idea further by making the representation typed:

```
BranchDigest {
    objective
    hypotheses_tried
    important_decisions
    changed_components

    candidate_artifacts
    evidence_gained
    counterexamples_found
```

```
    failures
    explanations_of_failures

    unresolved_questions
    likely_next_moves

    confidence_and_limits

    source_state
    source_artifacts[]
}
```

Then a branch can be compared without shoving its entire 100,000-token history into another context.

There should be multiple compression resolutions:

```
full retained artifacts
        ↓
EvidenceSummary
        ↓
BranchDigest
        ↓
StatusNarration
        ↓
Headline
```

But this is **progressive compression, not destructive compression**.

The original evidence survives.

The summary can be regenerated.

The summary can be wrong.

Acceptance never silently becomes dependent on a lossy summary when the underlying test evidence is available.

### “Thinking” should be split into several types

I would ban one generic `thinking` field from BW.

There are at least these distinct objects:

```
NativeReasoningTrace
NativeReasoningSummary
PromptedAnalysis
Hypothesis
Critique
DecisionRationale
DisplayActivity
```

They do not have the same semantics.

A native reasoning stream—if an inference backend actually exposes one—is retained as whatever that backend contract says it is.

A reasoning summary is not mislabeled as the full trace.

An explicit architectural analysis is just another generated artifact.

“Thinking…” in the UI is merely a projection that a generation is in flight.

And most importantly:

> **Correct recovery cannot depend upon hidden chain-of-thought.**

Everything BW itself needs to continue must be in explicit state, results, evidence, or a qualified model snapshot.

### Tool calls are typed effect proposals

The tool architecture should also not inherit an API-provider ontology.

The semantic object is not:

```
assistant_message.tool_calls[0]
```

It is:

```
EffectProposal<Action> {
    action
    exact_inputs
    claimed_purpose
    required_output_type
}
```

The runtime derives the actual capabilities needed, validates policy and resource preconditions, freezes the operation, records intent, executes it, and produces:

```
Observation<Result>
```

That observation has provenance tying it to:

```
computation
source world
executable
arguments
environment
capabilities
receipt
actual output artifacts
```

A model can emit perfectly valid JSON claiming:

```
{"tests_passed": true}
```

and still possess nothing more than:

```
Claim<TestReport>
```

The real process execution produces:

```
Observation<TestReport>
```

or stronger:

```
Evidence<TestReport>
```

if its preconditions and evaluator contract make it valid evidence.

That distinction between **shape, provenance, commitment, and authority** is one of the type system's most important jobs.

## Verification should have its own branches

A test-time-compute-optimal coding runtime should not spend all extra compute asking solvers to solve harder.

Some of the most valuable compute should be spent trying to prove that the apparent solution is wrong.

AdverMCTS is particularly relevant: it couples a solver search with an attacker that generates targeted tests intended to expose logical divergences among candidate programs, explicitly addressing “pseudo-correct” code that passes visible tests. [13] Fine-grained verification work similarly shows the benefit of decomposing correctness into explicit checkable criteria rather than relying on a single holistic score. [14] Counterexample-guided program-verification research provides the same broader lesson: failed checks become useful state for refinement rather than merely a binary rejection. [15]

So BW should naturally build trees like:

```
candidate-A
├── implementation continuation
├── static verifier
├── regression verifier
├── adversarial-test generator
│   ├── attack-1
│   ├── attack-2
│   └── attack-3
├── concurrency falsifier
└── comparator vs candidate-B
```

This leads to a useful conceptual distinction:

```
SOLVER COMPUTE
    improves a candidate

VERIFIER COMPUTE
    reduces uncertainty about a candidate

SEARCH COMPUTE
    decides where solver/verifier compute is useful

SYNTHESIS COMPUTE
    reuses information across branches
```

All four compete for the same global resource budget.

### Verification compute should also be adaptive

If A is obviously broken and B is obviously strong, spending ten independent judges comparing them is waste.

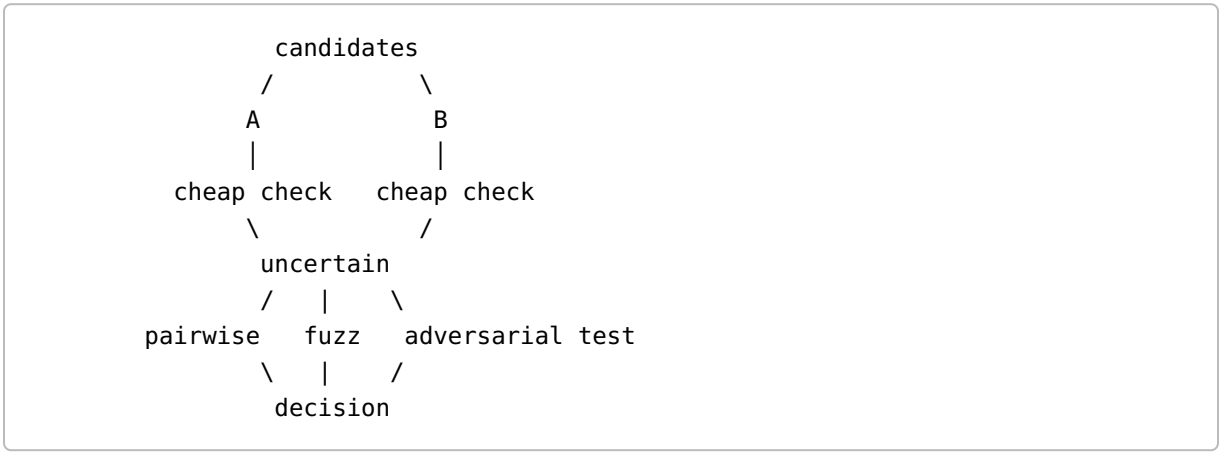
If A and B are almost tied, additional discriminative tests may have enormous decision value.

That means the same value-of-computation logic applies recursively to the verifier.

A likely late-stage tree looks less like:

```
generate
generate
generate
generate
generate
```

and more like:




Pairwise selection matters because point scoring can be poorly calibrated even when relative comparison is useful; the larger lesson from current verification research is to spend additional judge/test compute where the choice remains uncertain rather than evenly. [16]

### “Speculate first, commit once”

Research on risk-aware agentic test-time scaling explicitly separates simulated exploration from real-world commitment because real agent actions can be irreversible or costly. [17]

BW should make that an architectural invariant.

Classify effects:

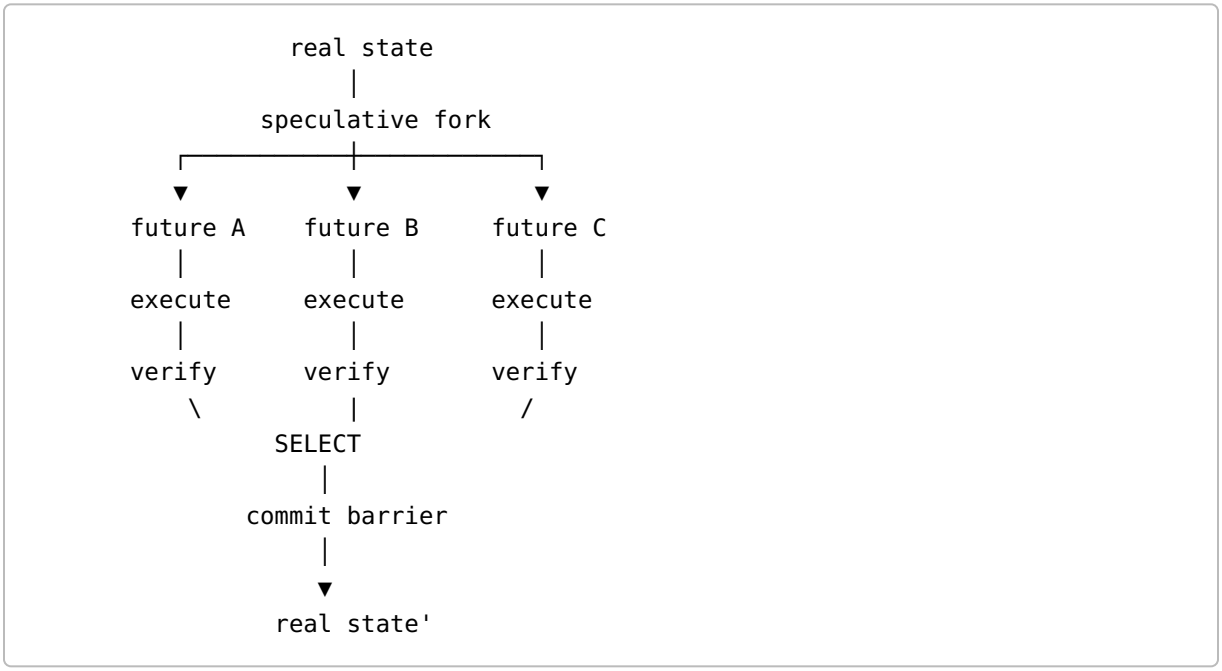
| Class | Meaning |
|---|---|
| `Pure` | No external mutation. |
| `BranchLocal` | Mutates only the branch's disposable world. |
| `ReconcilableExternal` | External mutation, but outcome can be established by receipt/ idempotency/CAS. |
| `IrreversibleExternal` | Cannot reliably be undone or reconciled. |

Test-time exploration should happen overwhelmingly in the first two classes.

In the ideal future architecture, even more of reality becomes branchable:

```
COW model state
COW filesystem
COW process memory
COW dependency caches
COW databases
branch-local services
virtualized sockets
transactional outbound network
staged deployment
staged messages
```

Then the pattern becomes:



That is the limiting ideal of BranchWrite.

**Do not undo bad effects. Make unselected effects never become shared reality in the first place.**

## Durability becomes simpler when readiness is derived

Your proposed recovery test is exactly right, but the graph formulation lets us make it even stronger.

The old interpretation would be:

> Child finished → save result → crash before result is delivered to parent → recover delivery exactly once.

The better invariant is:

> **There is no delivery operation to recover.**

Suppose:

```
Continuation K:
    requires Result<R> produced by computation C7
```

Before C7 completes:

```
K.ready = false
```

After the durable fact exists:

```
Completed(C7, result=R44)
```

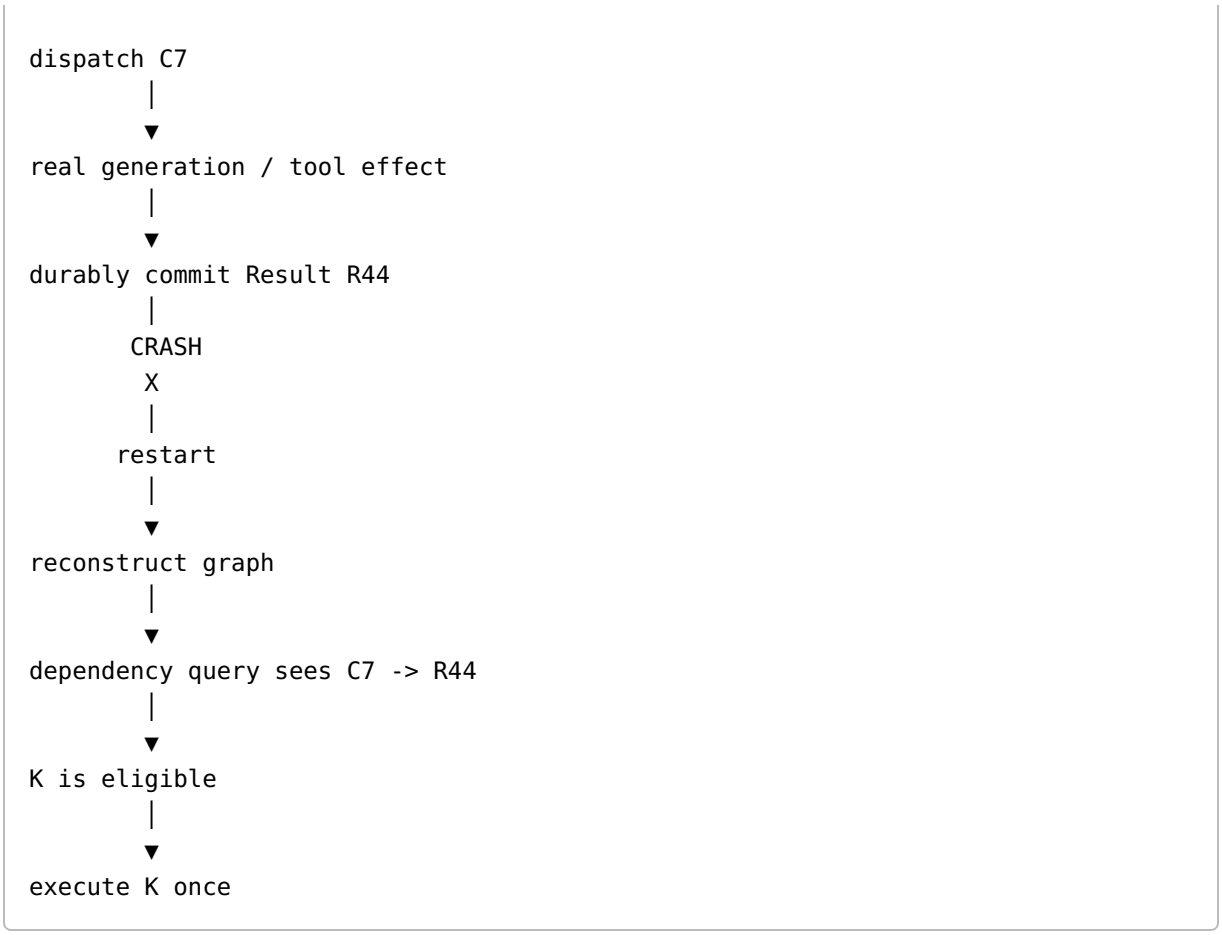
the frontier query deterministically yields:

```
K.ready = true
```

The runtime need not mutate K to tell it that the child returned.

That means the exact test becomes:

```
parent continuation K
        │
        └── requires result of C7
```




The assertions are:

**C7 is not dispatched again.**

**R44 is unchanged.**

**K consumes the existing R44.**

**K's semantic successor exists at most once.**

**C7 remains visible forever in history.**

**Nothing has to be "popped back onto" an in-memory stack.**

This borrows the strongest insight from durable-execution systems while avoiding an unnecessary limitation. Temporal, for example, rebuilds workflow state from a durable event history and reuses recorded activity results instead of re-executing those effects during replay. [18] BW can use the same principle of durable result reuse, but because its semantic object is already a persistent search graph, it need not pretend that the only thing being reconstructed is one deterministic procedural call stack.

### Use deterministic successor identity

A particularly clean implementation can derive a logical successor key from its semantic inputs:

```
successor_key =
    H(
      continuation_definition,
      source_state,
      committed_dependency_results,
      policy_semantics_version
    )
```

Repeated recovery can rediscover that the successor is ready, but it discovers the **same logical computation**, not a fresh one.

The dispatcher then has:

```
Planned
IntentDurable
Dispatched
ResultCommitted
```

for that stable identity.

This moves exactly-once semantics to the place where BW can actually provide them.

### Exactly-once must stop at the real effect boundary

We should still not say:

> “BW guarantees exactly-once tools.”

That is impossible in general.

For local BW state, we can make the following extremely strong:

> A durably completed computation will not be re-executed merely because the BW runtime crashed, and a committed result will not generate duplicate logical successor computations.

For external effects, the truth depends upon the sink.

A service with idempotency keys or queryable transaction receipts can support strong reconciliation.

A guarded Git ref update can support a conditional commit.

An arbitrary external side effect with no queryable identity may remain unknowable after connection loss.

BW v0.4 already gets this philosophical distinction right: uncertainty must remain visible rather than being papered over by retries. filecite turn0file0 The COW CLI applies the same rule to inference: exact request body, operation ID, and original revision are retained, and uncertain acknowledgement recovery uses that original operation rather than issuing a semantically new generation. filecite turn0file2

So the invariant I would put in the handoff is:

> **No completed computation is repeated solely because BW crashed. No durable result is incorporated into one logical successor more than once. No irreversible external effect is claimed exactly once unless the external system supplies enough semantics to make that statement true.**

### Crash testing should cover the whole graph

The key test you named should be mandatory, but it should live inside a full crash matrix:

```
before durable intent

after durable intent
before dispatch

after dispatch
before acknowledgment

during provisional model streaming

after remote generation/effect commit
before BW knows outcome

after BW result commit
before dependent computation runs     <-- your key case

after dependent computation becomes runnable
before its dispatch

after successor completes
before a multi-parent join

during candidate comparison

during integration

before publication
```

```
after publication
before local publication receipt
```

Recovery should follow from graph semantics.

If each one needs a bespoke “if crashed_here” branch in the implementation, the architecture is wrong.

## The implementation handoff

The implementation handoff I would give the builder is now materially different from the previous one.

### The mission statement

> **Build BW as a durable, typed, budgeted search runtime over copy-on-write cognitive and execution state.**
>
> The tree is not transcript visualization. A stack is not the durable state model. Branches are candidate futures. The runtime allocates test-time compute among generating, refining, observing, verifying, challenging, comparing, synthesizing, and committing those futures.

### The core object model

```
StateCapsule {
    id

    task_contract
    objective

    cognitive_state?
    world_state?
    knowledge_state

    capabilities
    unresolved_effects

    primary_lineage_parent?
    causal_parents[]

    typed_bindings

    budget_account
    policy_constraints

    semantic_status:
        frontier
        dormant
```

```
        terminal
        unresolved
}
```

Then:

```
Run {
    graph
    frontier
    incumbents
    budget
    search_policy
}
```

Search statistics should be **separate from semantic identity**:

```
SearchEstimate {
    state_or_computation

    estimated_success
    uncertainty
    novelty
    estimated_improvement
    estimated_information_gain

    marginal_token_cost
    marginal_prefill_cost
    wall_cost
    memory_cost
    verification_cost

    COW_locality
    estimated_risk

    estimator_version
}
```

That matters because changing a value heuristic should not mutate historical truth.

### The primitive computation algebra

I would make these primitives—not an endless list of endpoint-shaped operations:

```
EXPAND
CONTINUE

ACT
OBSERVE

TRANSFORM

VERIFY
CHALLENGE

COMPARE
SELECT

COMPOSE
COMMIT

ABANDON
```

Existing BW verbs remain an excellent human-facing DSL:

```
ASK   -> EXPAND / CONTINUE / TRANSFORM / COMPARE
PLAN  -> freeze proposed computation against exact state
DO    -> ACT / COMMIT
LOOK  -> OBSERVE
SAVE  -> pin/retain named result
CHECK -> VERIFY / CHALLENGE / COMPARE / SELECT
STOP  -> ABANDON / terminal disposition
```

And the structural forms remain useful:

```
FLOW
REPEAT
BRANCH
PARALLEL.PLAN
PARALLEL.DO
ON
```

But internally those are graph constructors and policy constraints.

I would **not** make `PARALLEL.DO` a separate code path.

It means:

```
several eligible ACT computations
+
independent branch-local world scopes
+
scheduler allowed to dispatch them concurrently
```

Likewise `PARALLEL.PLAN` means several model expansions sharing an immutable base while keeping subsequent context histories independent.

### The modules

Responsibility boundaries, not plugin frameworks:

```
branchwrite/
    types/
        semantic types
        provenance
        capabilities

    graph/
        state capsules
        computations
        dependency predicates
        causal edges
        joins

    runtime/
        transactional state
        readiness derivation
        dispatch
        recovery

    search/
        frontier
        budgets
        incumbents
        cost models
        value-of-computation
        novelty
        policies

    cow/
        persistent model-context backend
        fork / rewind / continue
        operation recovery
```

```
    COW cost/locality telemetry

worlds/
    execution snapshots
    workspaces
    process/service ownership

effects/
    file
    process
    network
    repository
    external commit handlers

verify/
    deterministic checks
    adversarial checks
    comparison
    selection
    acceptance contracts

derive/
    headline
    status narration
    branch digest
    evidence summary
    explicit reasoning artifacts

integrate/
    multi-parent synthesis
    candidate integration
    publication

policy/
    authorization
    human decisions
    resource limits

ui/
    tree projection
    frontier projection
    evidence graph
    incumbent view
    resource view
    budget view

benchmark/
    deterministic search fixtures
```

```
live COW qualification
recovery fault injection
test-time compute curves
```

### Implementation ordering

The order should be driven by invariants, not visible features.

**First: semantic graph.**

Implement typed values, provenance, immutable state capsules, computation identities, typed dependency predicates, result commitment, and frontier reconstruction.

Then prove your crash case.

Do **not** build autonomous search before this passes.

**Second: native persistent model state.**

Bind actual COW operations. Preserve active-turn continuation, provisional/committed output distinction, revision checks, fork semantics, and original-operation recovery exactly as provided by the native service. filecite turn0file2

**Third: branchable execution worlds.**

Real files. Real programs. Real tests. Real process/resource identities. Honest isolation boundaries.

**Fourth: evidence and falsification.**

Independent verification branches, protected acceptance criteria, targeted counterexamples, pairwise comparison.

**Fifth: search policy.**

Start with controlled policies:

```
linear
best-of-N
uniform tree
adaptive tree
adaptive tree + verification allocation
adaptive tree + COW locality cost
population / resampling
```

That is important because BW should be able to experimentally prove whether its fancy scheduler actually buys anything.

**Sixth: synthesis and commitment.**

Explicit multi-parent synthesis, integration worlds, post-integration verification, guarded publication.

**Seventh: UX generators.**

Headlines, summaries, status narration, branch digests.

Those come late precisely because they should never accidentally become the authoritative state layer.

### What the first convincing demonstration should do

Not:

```
prompt
→ model edits file
→ tests pass
```

Instead, choose a repository task where there are at least two plausible approaches and one deceptively plausible wrong solution.

The run must:




Then:

1. Generate genuinely different hypotheses.
2. Spend **unequal** compute on them.
3. Execute implementations in isolated branch worlds.
4. Generate typed branch digests.

5. Maintain the current incumbent.
6. Create independent verifier/falsifier branches.
7. Produce at least one targeted test intended to distinguish candidates rather than merely rerun a generic suite.
8. Allow one weak branch to be abandoned because its marginal expected value is poor.
9. Crash after a real child result is durably committed but before its dependent continuation executes.
10. Restart and derive the continuation as runnable from the retained result.
11. Prove the child generation/effect was not repeated.
12. Compare or synthesize the surviving branches explicitly.
13. Verify the exact selected/integrated candidate.
14. Cross exactly one controlled commit/publication boundary.
15. Reconstruct the tree, knowledge graph, evidence, resources, and budget from persistent state.

Then run the **same task** at several budgets.

The expected output should not merely be:

```
budget 1 = worse answer
budget 2 = better answer
```

It should show different search morphology:

```
small budget
    2 branches
    shallow verification
    early selection

medium budget
    4 branches
    one adversarial verifier
    pairwise comparison

large budget
    6 initial branches
```

```
    3 quickly abandoned
    2 refined
    1 counterexample-driven repair
    deeper independent verification
```

That demonstrates that “more test-time compute” means **more intelligently allocated search**, not a longer conversation.

### The research benchmark should compare policies at equal cost

This is critical.

Measure:

| Policy | Why it exists |
|---|---|
| Linear single trajectory | Baseline. |
| Uniform best-of-N | Measures pure parallel sampling. |
| Uniform tree | Measures branching without adaptive allocation. |
| Adaptive budget-aware tree | Measures dynamic expand/refine behavior. |
| Adaptive + verifier allocation | Measures falsification and selection compute. |
| Adaptive + COW-aware cost | Measures whether physical tree reuse translates to real efficiency. |

Report final accepted-task rate, task utility/simple regret where ground truth permits it, wall time, model tokens, prefill/shared-prefix work, memory pressure, number and depth of branches, amount of compute spent on ultimately rejected branches, verifier accuracy, counterexamples found, and crash-recovery overhead.

The literature gives good reason not to assume “more branches = better”: current work finds strong gains from dynamic search and trajectory reuse, but also emphasizes that allocation strategy and verifier quality determine whether extra compute helps at all. [19]

### The invariants I would make non-negotiable

The full handoff has a longer acceptance program, but these are the architectural core:

| Invariant | Meaning |
|---|---|
| **Immutable past** | Completed history is never rewritten to make the run cleaner. |
| **Derived frontier** | Runnable work is reconstructable from durable semantic facts. |
| **No hidden return delivery** | Parent continuation never depends on an in-memory result slot. |

| Invariant | Meaning |
|---|---|
| **Committed computation reuse** | A completed child is not regenerated because BW restarted. |
| **Explicit uncertainty** | Unknown external outcomes remain unknown. |
| **Evidence over assertion** | Generated prose cannot substitute for observed acceptance evidence. |
| **Branch-local speculation** | Unselected candidate mutation stays isolated. |
| **Commit barrier** | Irreversible shared-world effects happen only through explicit commit. |
| **COW-native cognition** | Persistent context is used as state, not flattened into reconstructed chat requests. |
| **Explicit knowledge joins** | Cross-branch reuse happens through recorded artifacts/synthesis, not invisible history leakage. |
| **Budget-aware search** | Different branches may receive radically different compute. |
| **Marginal-cost accounting** | Prefix reuse and real resource cost matter, not tokens alone. |
| **Independent falsification** | Verifier branches may actively attack solver candidates. |
| **Annotations lack authority** | Headline/summary/status generators cannot advance workflow state. |
| **Anytime incumbent** | BW always knows its best currently justified candidate. |
| **One semantic controller** | Models, tools, adapters, generators, and UI do not secretly run their own workflows. |

## What “100 years in the future” changes

Taking that premise seriously leads to a more ambitious endpoint than simply durable agents.

Imagine the primitives are perfect.

A model context can be forked in microseconds with essentially free shared state.

A process tree, filesystem, database, dependency cache, and local service topology can also be forked as one coherent execution future.

Inference hardware understands the whole branch topology and can decode hundreds of siblings while physically storing the shared prefix once.

External writes can be transactionally escrowed.

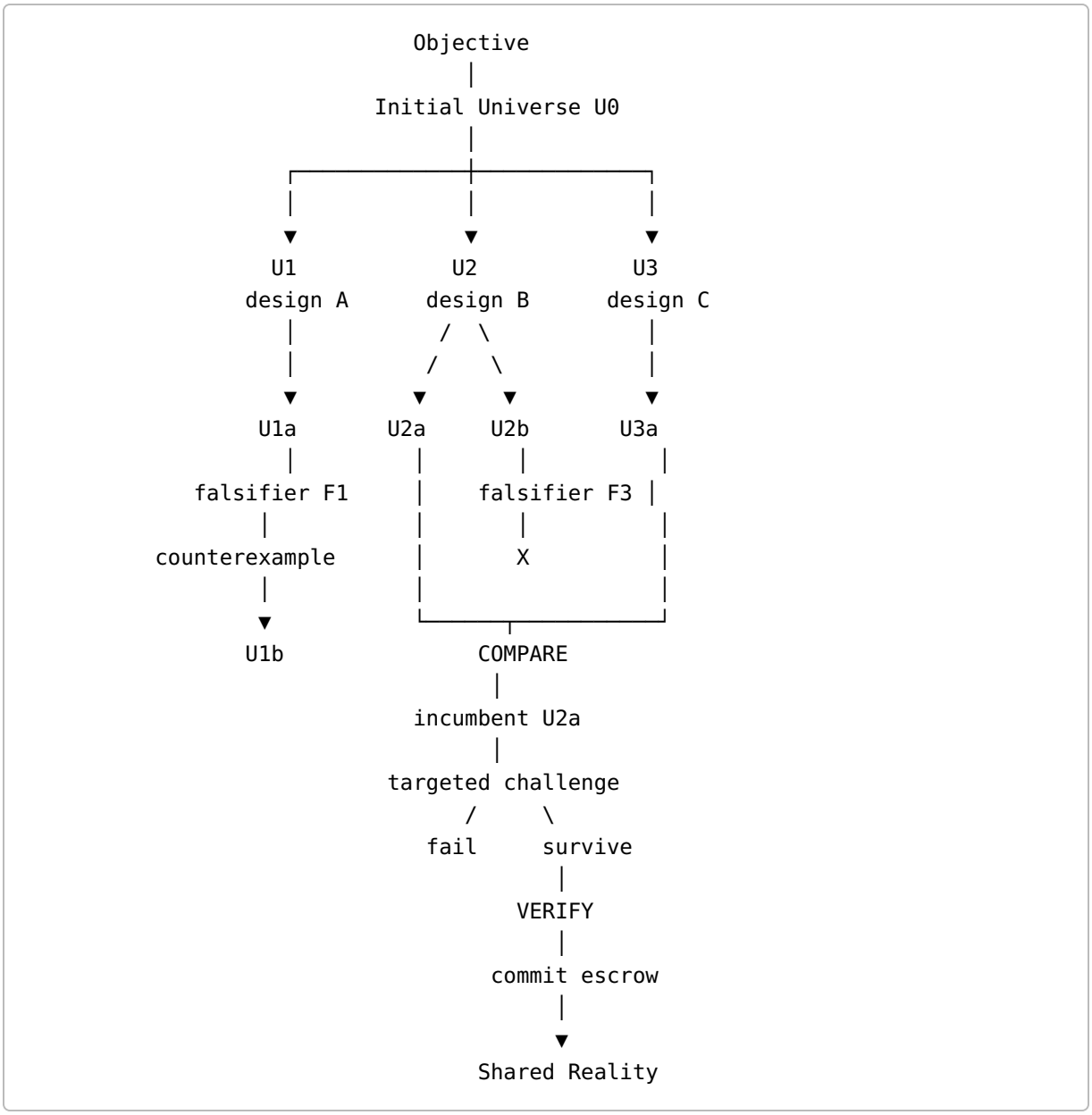
An execution can produce a machine-checkable provenance object tying every output byte to its exact inputs and environment.

Continuation state is natively durable.

Tests can fork reality itself.

Then BW does not look like an API client at all.

It looks like this:




Every branch is a **counterfactual future**.

The runtime is continuously asking:

> Which counterfactual is worth making more detailed?
>
> Which one is worth trying to falsify?
>
> Which two are worth comparing?
>
> Which discovery from a losing future should inform another future?
>
> Which computations are redundant because these states are effectively the same?
>
> Is the current incumbent already good enough that the expected value of another branch is below its compute price?
>
> Has sufficient evidence accumulated that crossing the commit barrier is rational?

That is the real theoretical potential of your COW tree.

And it reveals one final design principle I think is important:

**The ideal search object is not actually the conversation. It is the future state of the task.**

Model context is one component of that future.

Code is another.

Evidence is another.

Processes and services are another.

The budget spent learning about the future is another.

Your existing architecture already recognized that model rewind does not rewind external reality and that task branches, COW branches, and worktrees are distinct objects. filecite turn0file0 The next step is to push that distinction all the way through the runtime and make the **candidate future**—not the chat turn, API call, or stack frame—the fundamental unit of computation.

That also gives BW a clean answer to the classical-agent problem:

> It is not an LLM taking turns with tools.

It is a search system whose state space happens to contain branchable language-model cognition, branchable executable worlds, accumulated evidence, and eventually a controlled path back into shared reality.

**That is the architecture I would build.**

Download `BranchWrite_TTC_Optimal_Handoff_v1.0.md`

---


1 3 19 Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters
https://arxiv.org/abs/2408.03314

2 Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models
https://proceedings.mlr.press/v235/zhou24r.html?utm_source=chatgpt.com

4 Aligning Tree-Search Policies with Fixed Token Budgets in Test-Time Scaling of LLMs
https://proceedings.mlr.press/v306/miyamoto26a.html?utm_source=chatgpt.com

5 HyPER: Bridging Exploration and Exploitation for Scalable LLM Reasoning with Hypothesis Path Expansion and Reduction
https://proceedings.mlr.press/v306/qiu26j.html?utm_source=chatgpt.com

6 Scaling Test-Time Compute for Agentic Coding
https://arxiv.org/abs/2604.16529?utm_source=chatgpt.com

7 MCTS Based on Simple Rerget | Proceedings of the International Symposium on Combinatorial Search
https://ojs.aaai.org/index.php/SOCS/article/view/18221?utm_source=chatgpt.com

8 Agentic Test-Time Scaling for WebAgents
https://arxiv.org/abs/2602.12276?utm_source=chatgpt.com

9 SGLang: Efficient Execution of Structured Language Model Programs
https://arxiv.org/abs/2312.07104

10 Programming with Algebraic Effects and Handlers
https://arxiv.org/abs/1203.1539

11 StructuredTaskScope (Java SE 28 & JDK 28 [ad-hoc build])
https://cr.openjdk.org/~alanb/sc-jdk28/api/java.base/java/util/concurrent/StructuredTaskScope.html?utm_source=chatgpt.com

12 Graph of Thoughts: Solving Elaborate Problems with Large Language Models
https://arxiv.org/abs/2308.09687?utm_source=chatgpt.com

13 AdverMCTS: Combating Pseudo-Correctness in Code Generation via Adversarial Monte Carlo Tree Search
https://proceedings.mlr.press/v306/li26dt.html?utm_source=chatgpt.com

14 16 FineVerify: Scaling Test-Time Compute with Fine-Grained Self-Verification for Agentic Search
https://arxiv.org/abs/2606.00660?utm_source=chatgpt.com

15 ExVerus: Verus Proof Repair via Counterexample Reasoning
https://proceedings.mlr.press/v306/yang26bt.html?utm_source=chatgpt.com

17 ARTIS: Agentic Risk-Aware Test-Time Scaling via Iterative Simulation
https://arxiv.org/abs/2602.01709?utm_source=chatgpt.com

18 Temporal Workflow | Temporal Documentation
https://docs.temporal.io/workflows?utm_source=chatgpt.com